

MINI ASSIGNMENT

Name: Kovvuri Charan Sai

Register No.: 99230040353

Department: CSE (AI & ML)

Date: 11-09-2026

1. How should an application handle an HTTP 429 Too Many Requests response?

- A 429 response generally indicates that the server is temporarily limiting the number of requests from the client.
- The program should first check for a **Retry-After** header. If the server provides one, the application should wait for that amount of time before trying again.
- If Retry-After is not present, the application can use **exponential backoff**, increasing the delay between retries. A small random jitter can be added so multiple clients do not retry simultaneously.
- Retries should always have a maximum attempt count. After that limit, the program should stop retrying, record the failure, and handle it as an unsuccessful request.
- The request should only be retried when repeating the operation is safe and will not create an unwanted duplicate action.

Example: If the server returns 429 with Retry-After: 20, the client should wait about 20 seconds before attempting the request again.

Key Point: Treat 429 as a temporary rate-limit condition and use controlled, respectful retries.

2. What is a safe way to process an API response when its body may not contain valid JSON?

- Calling **response.json()** assumes that the response body contains valid JSON. The call can fail if the server returns HTML, plain text, malformed JSON, or an empty body.
- The JSON conversion should therefore be placed inside a **try-except** block so a parsing error does not unexpectedly stop the application.
- Before parsing, checking the HTTP status code and Content-Type can help determine whether JSON should be expected.
- If parsing fails, the program should log useful diagnostic information and then either use a safe fallback value or raise an application-specific error.

Example:

```
try:
    data = response.json()
except json.JSONDecodeError:
    logger.error("Invalid JSON received")
    data = None
```

Key Point: API responses should be treated as potentially invalid, even when JSON is normally expected.

3. When should a Python program use dictionary .get() instead of bracket notation?

- For a dictionary, **data['key']** expects the requested key to exist. If it is absent, Python raises a **KeyError**.
- The **data.get('key')** method is safer when the field is optional because it returns **None** if the key does not exist.
- A custom fallback can also be supplied, for example **data.get('phone', 'Not provided')**.

- This is useful for API responses because optional fields may be missing, different response versions may contain different fields, and error responses may not have the same structure as successful responses.
- Bracket notation is appropriate when the key is mandatory and its absence should immediately be treated as an error.

Example:

```
data = {"name": "Alice"}  
data["phone"] # KeyError  
data.get("phone") # None  
data.get("phone", "N/A") # "N/A"
```

Key Point: Use `.get()` for optional or uncertain fields; use `[]` when the key is required.